

Computer Networks LAB

LAB 3 – GUI Based Multi-Client Chat Server

Aravinda Kannan KS (24BCE1290)

S Tharun Parykshyt (24BCE1119)

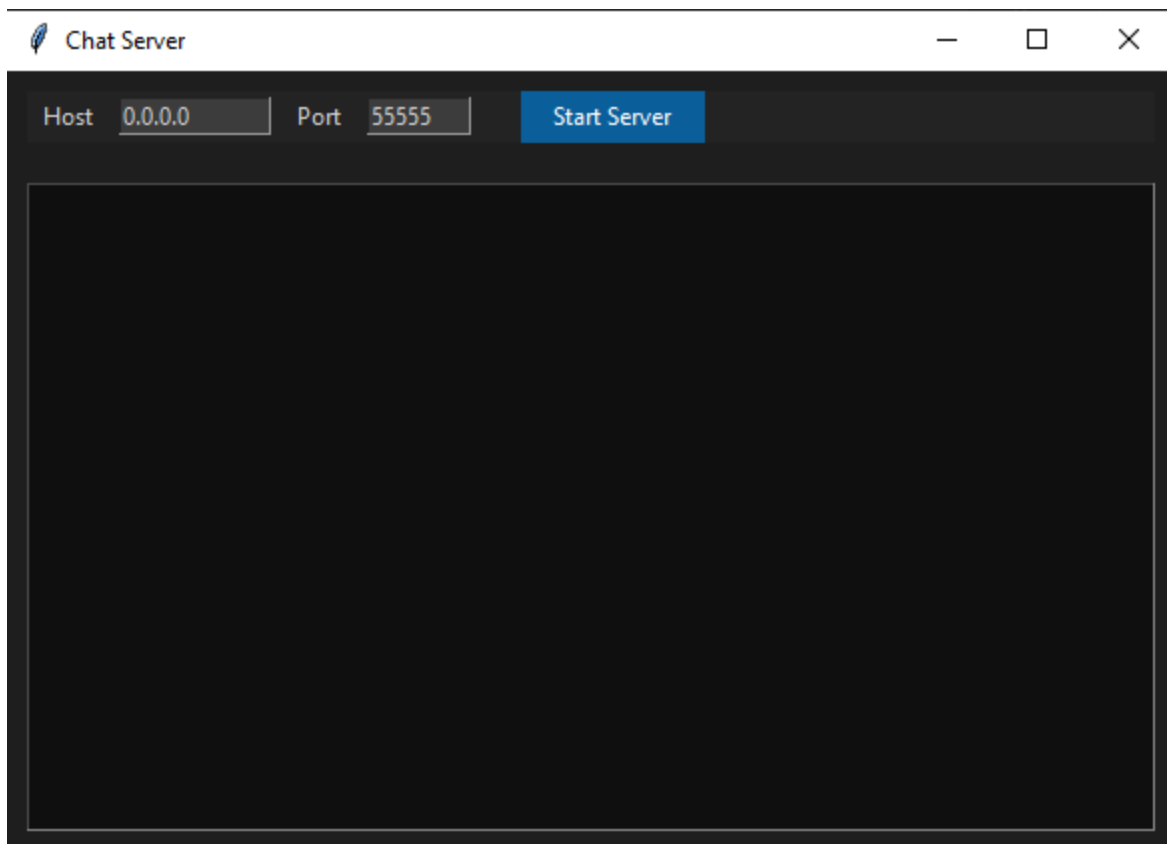
GUI Based Multi Chat Server

This experiment demonstrates a **multi-client chat application** built using **TCP sockets** with a **GUI-based server and clients**.

Multiple users connected on the same Wi-Fi network can communicate simultaneously, with support for:

- Global (broadcast) messages
 - One-to-one private messages
 - Dynamic join and leave handling
 - Real-time server monitoring through a GUI
-

Server GUI



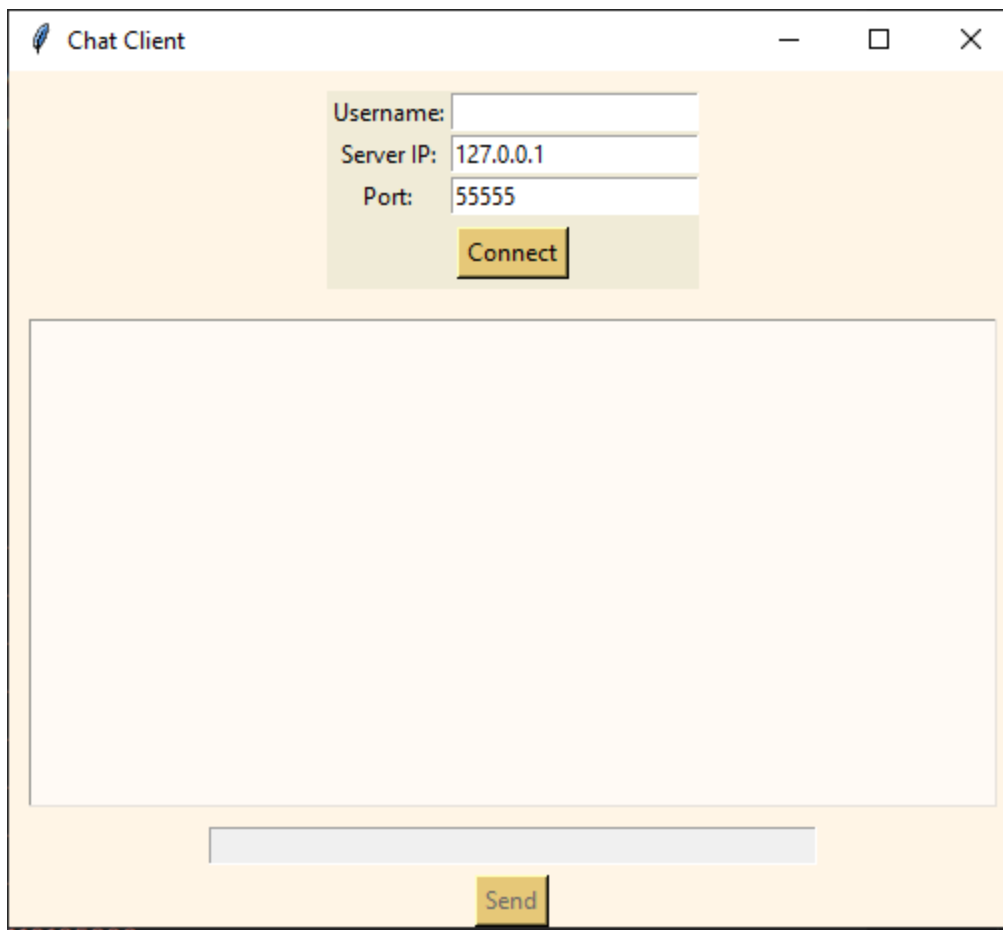
The **Server GUI** acts as a **control interface** for the chat server.

Its responsibilities are intentionally limited to:

- Taking host and port input
- Starting and stopping the server
- Displaying real-time server logs

All networking and message routing logic is delegated to the backend server class.
This separation ensures the GUI remains responsive even when multiple clients are connected.

Client GUI

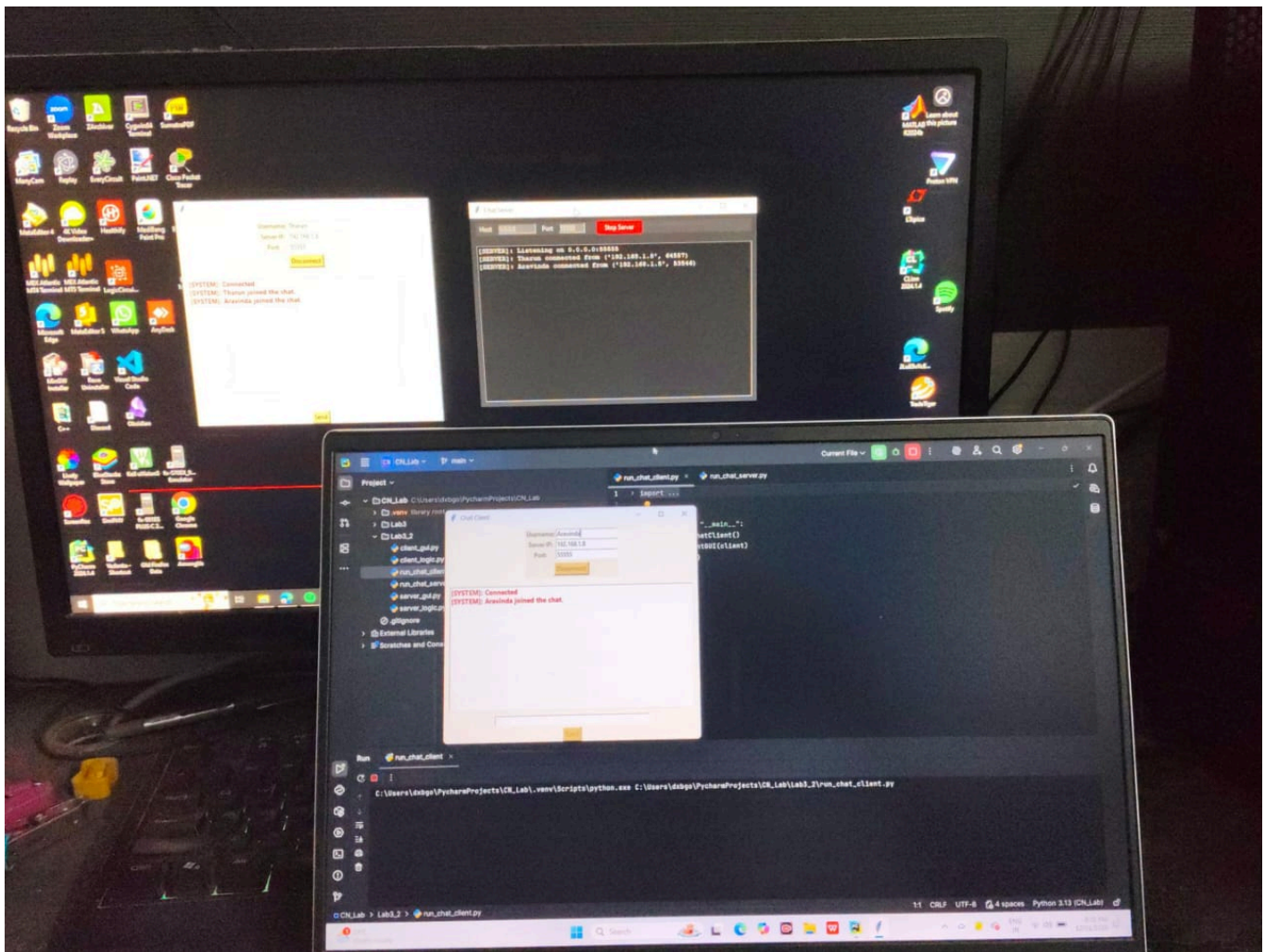


The **Client GUI** is responsible for:

- Collecting user credentials (username, IP, port)
- Displaying incoming and outgoing messages
- Differentiating message types using color coding
- Handling private and public messages cleanly

Socket operations are never executed directly on the UI thread, preventing freezes or unresponsive behavior.

Setup

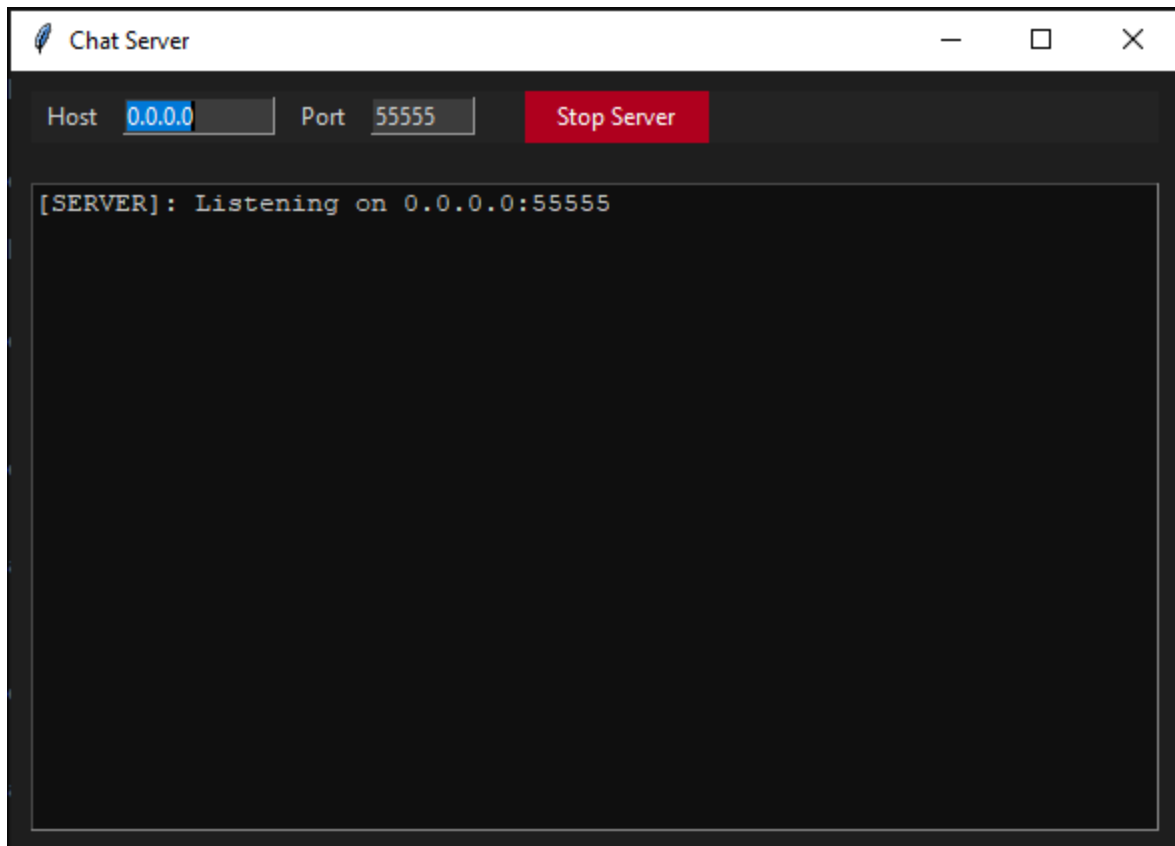


Network setup used during testing:

- Aravinda Kannan: 2 client instances
- S Tharun Parykshyt: 1 server + 1 client
- All devices connected to the same Wi-Fi network
- IP discovered using `ipconfig`
- Server hosted on `0.0.0.0` to accept LAN connections
- IP used: `192.168.1.8`
- Port used: `55555` (non-reserved, safe port)

Devices connected on the same network were able to communicate without additional configuration.

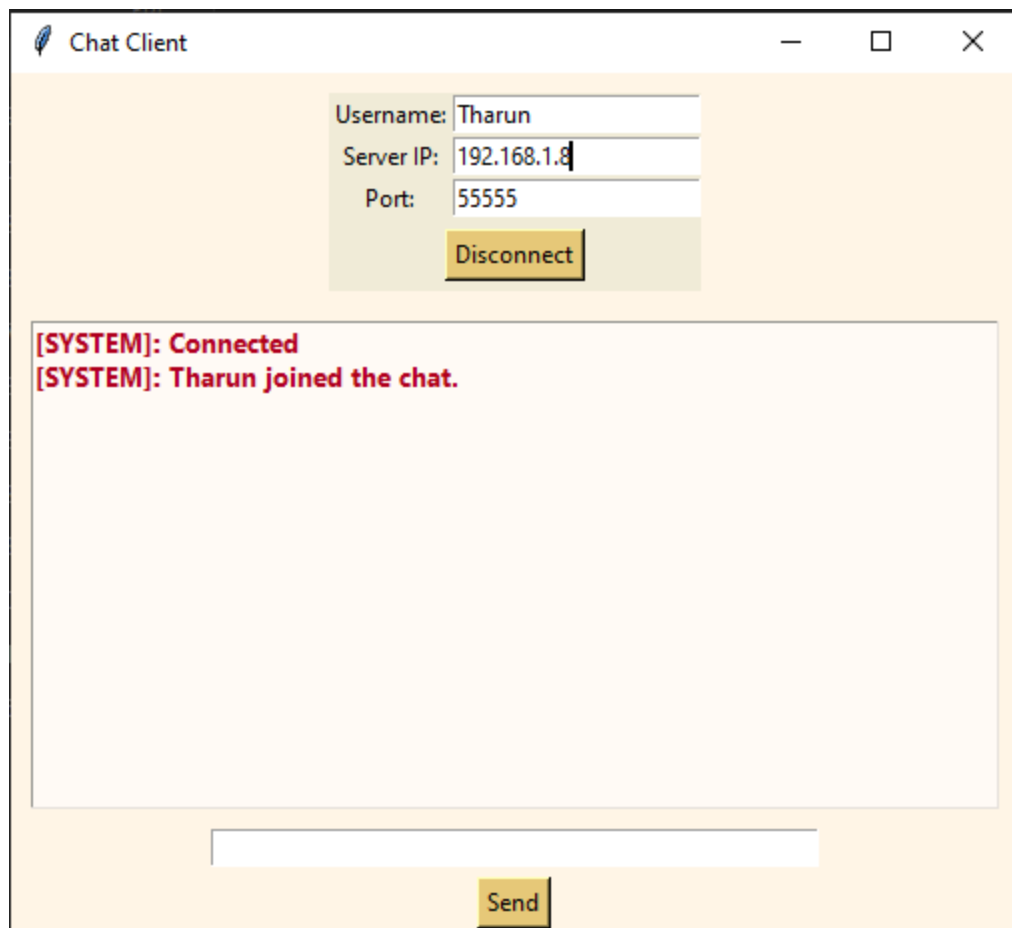
Server Boots Up



Once started, the server begins listening on the specified IP and port and waits for incoming client connections.

Client Join Sequence

Tharun Joins



Aravinda Joins from Two Devices

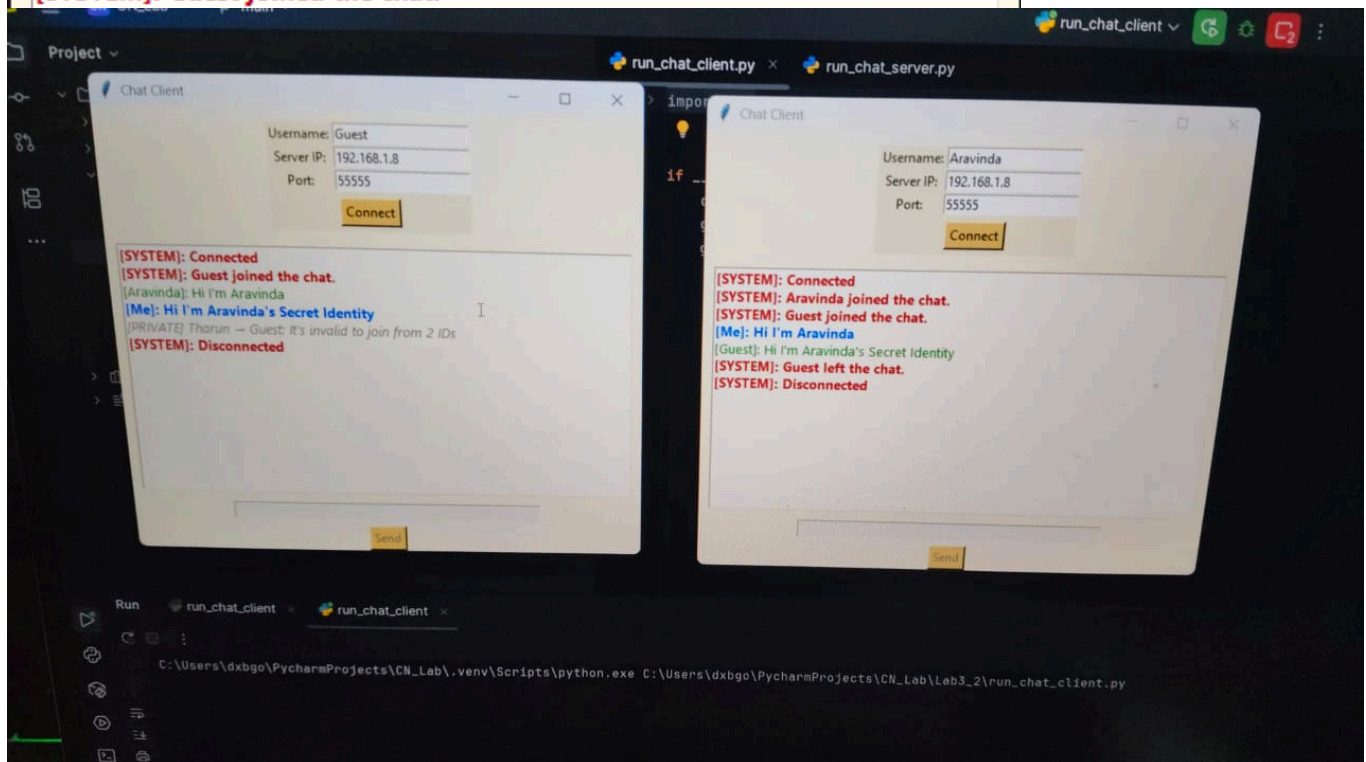
Chat Client

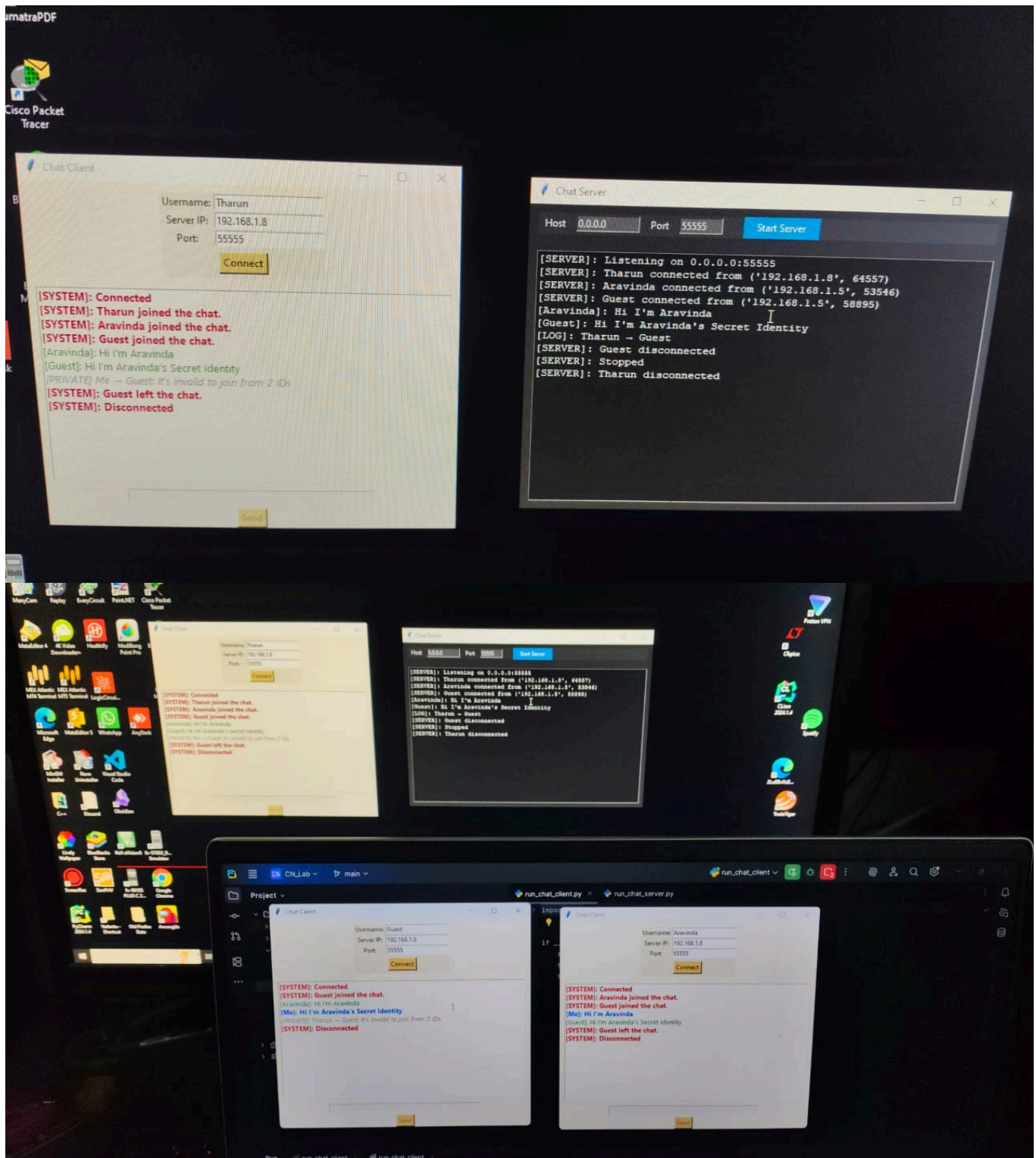
Username:

Server IP:

Port:

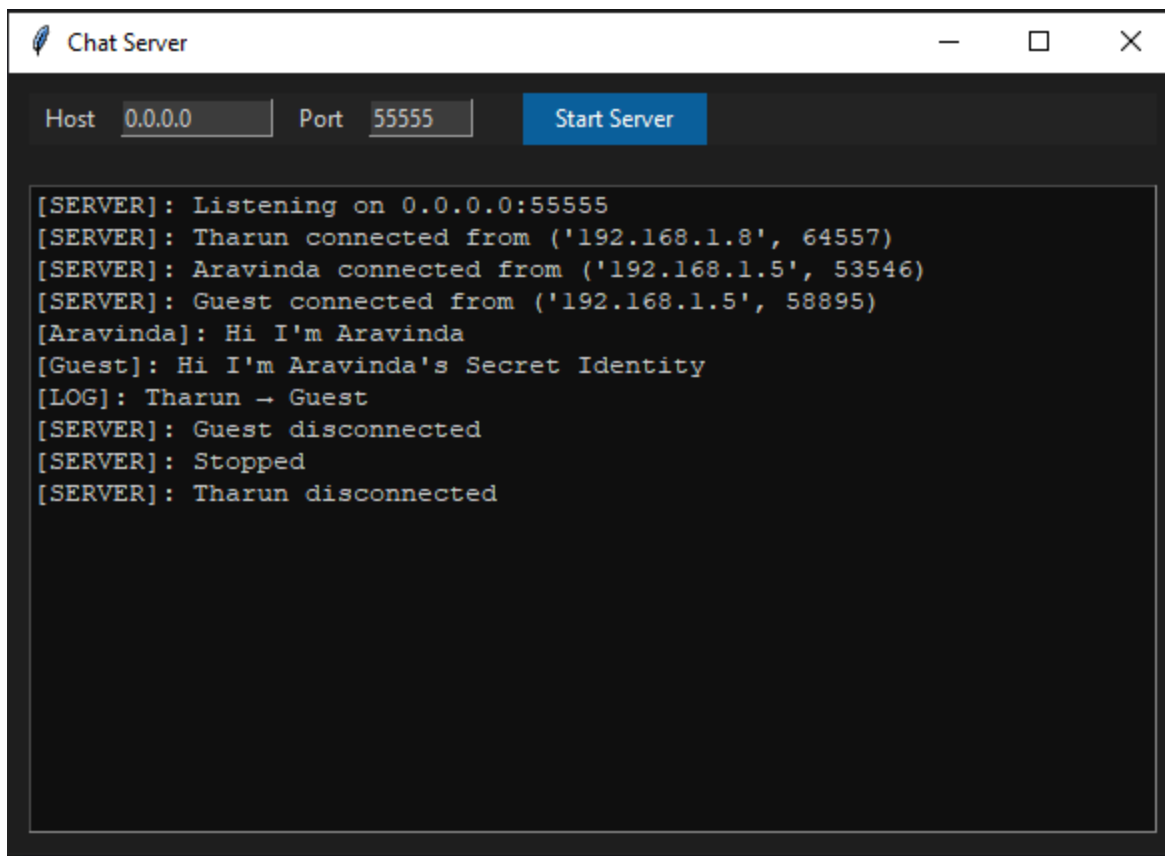
[SYSTEM]: Connected
[SYSTEM]: Tharun joined the chat.
[SYSTEM]: Aravinda joined the chat.
[SYSTEM]: Guest joined the chat.





Each client is identified uniquely using its username.
The server logs each join event and notifies other connected users.

Terminal Log



Observed behavior:

- Multiple users can chat simultaneously
- Private messages are routed only to the intended recipient
- Private messages are logged by the server but not broadcast
- Clients can join or leave without interrupting others
- Server remains stable throughout dynamic connections

Code Section

Server GUI

```
# =====  
# SERVER GUI (Tkinter)  
# =====  
  
import tkinter as tk  
import threading  
from typing import Optional
```

```

class ServerGUI:
    """
    GUI layer for the chat server.
    This file is ONLY responsible for:
    - User interaction
    - Displaying logs
    - Starting / stopping the server

    IMPORTANT:
    - No socket logic is written here
    - The server logic runs in a separate thread
    """

    def __init__(self, server) -> None:
        # Reference to the backend server object
        self.server = server

        # Thread in which the server will run
        self.server_thread: Optional[threading.Thread] = None

        # GUI-level running flag (not socket state)
        self.running = False

        # ---- Dark Theme Colors (UI only) ----
        BG = "#1e1e1e"
        PANEL = "#252526"
        FG = "#d4d4d4"
        BTN = "#0e639c"
        ENTRY_BG = "#3c3c3c"

        # Create main window
        self.root = tk.Tk()
        self.root.title("Chat Server")
        self.root.configure(bg=BG)

        # Top control panel
        frame = tk.Frame(self.root, bg=PANEL)
        frame.pack(pady=10, fill="x", padx=10)

        # Host input
        tk.Label(frame, text="Host", bg=PANEL, fg=FG).pack(side="left", padx=5)
        self.host_entry = tk.Entry(frame, bg=ENTRY_BG, fg=FG, width=12)
        self.host_entry.insert(0, "0.0.0.0") # Listen on all interfaces
        self.host_entry.pack(side="left", padx=5)

        # Port input

```

```

tk.Label(frame, text="Port", bg=PANEL, fg=FG).pack(side="left", padx=5)
self.port_entry = tk.Entry(frame, bg=ENTRY_BG, fg=FG, width=8)
self.port_entry.insert(0, "55555") # Non-reserved port
self.port_entry.pack(side="left", padx=5)

# Start / Stop button
self.start_btn = tk.Button(
    frame,
    text="Start Server",
    command=self.toggle_server,
    bg=BTN,
    fg="white",
    relief="flat",
    padx=12
)
self.start_btn.pack(side="left", padx=20)

# Log window
self.log = tk.Text(
    self.root,
    height=20,
    width=70,
    bg="#111111",
    fg=FG,
    insertbackground="white",
    state="disabled"
)
self.log.pack(padx=10, pady=10)

# Thread-safe logging
def write_log(self, msg: str) -> None:
    # Tkinter is NOT thread-safe
    # root.after schedules UI update on main thread
    self.root.after(0, self._append_log, msg)

def _append_log(self, msg: str) -> None:
    self.log.config(state="normal")
    self.log.insert("end", msg + "\n")
    self.log.see("end")
    self.log.config(state="disabled")

# Start / Stop server
def toggle_server(self) -> None:
    if not self.running:
        try:
            host = self.host_entry.get().strip()

```

```

        port = int(self.port_entry.get().strip())
    except ValueError:
        self.write_log("[ERROR]: Invalid host or port")
        return

    # Server runs in a background thread
    self.server_thread = threading.Thread(
        target=self.server.start,
        args=(host, port, self.write_log),
        daemon=True
    )
    self.server_thread.start()

    self.running = True
    self.start_btn.config(text="Stop Server", bg="#B00020")
else:
    self.server.stop()
    self.running = False
    self.start_btn.config(text="Start Server", bg="#0e639c")
    self.write_log("[SERVER]: Stopped")

def start(self) -> None:
    self.root.mainloop()

```

Client GUI

```

# =====
# CLIENT GUI (Tkinter)
# =====

import tkinter as tk
from tkinter import messagebox
import threading

class ClientGUI:
    """
    GUI layer for chat client.

    Responsibilities:
    - Collect user input
    - Display messages
    - Never touch sockets directly
    """

```

```

def __init__(self, client) -> None:
    self.client = client
    self.connected = False

    # ---- Colors ----
    BG, PANEL = "#FFF8E7", "#F3EED9"
    FG = "#4A3F2A"
    BTN = "#E6C97A"

    self.root = tk.Tk()
    self.root.title("Chat Client")
    self.root.configure(bg=BG)

    frame = tk.Frame(self.root, bg=PANEL)
    frame.pack(pady=10)

    # Username input
    tk.Label(frame, text="Username:", bg=PANEL).grid(row=0, column=0)
    self.user_entry = tk.Entry(frame)
    self.user_entry.grid(row=0, column=1)

    # Server IP
    tk.Label(frame, text="Server IP:", bg=PANEL).grid(row=1, column=0)
    self.host_entry = tk.Entry(frame)
    self.host_entry.insert(0, "127.0.0.1")
    self.host_entry.grid(row=1, column=1)

    # Port
    tk.Label(frame, text="Port:", bg=PANEL).grid(row=2, column=0)
    self.port_entry = tk.Entry(frame)
    self.port_entry.insert(0, "55555")
    self.port_entry.grid(row=2, column=1)

    # Connect / Disconnect
    self.connect_btn = tk.Button(
        frame, text="Connect", command=self.toggle_connection, bg=BTN
    )
    self.connect_btn.grid(row=3, columnspan=2, pady=5)

    # Output area
    self.output = tk.Text(
        self.root, height=15, width=60, state="disabled", bg="#FFDF6"
    )
    self.output.pack(padx=10, pady=5)

```

```

# Message input
self.msg_entry = tk.Entry(self.root, width=50, state="disabled")
self.msg_entry.pack(pady=5)
self.msg_entry.bind("<Return>", lambda _: self.send_msg())

self.send_btn = tk.Button(
    self.root, text="Send", command=self.send_msg, state="disabled", bg=BTN
)
self.send_btn.pack()

def write_log(self, msg: str) -> None:
    self.output.config(state="normal")
    self.output.insert("end", msg + "\n")
    self.output.see("end")
    self.output.config(state="disabled")

# Background listener thread
def listen_loop(self) -> None:
    while self.connected:
        msg = self.client.receive()
        if msg is None:
            break
        self.write_log(msg)
    self.on_disconnect()

# Send message
def send_msg(self) -> None:
    msg = self.msg_entry.get().strip()
    if not msg:
        return

    self.msg_entry.delete(0, "end")
    self.write_log(f"[Me]: {msg}")

    try:
        self.client.send(msg)
    except:
        self.on_disconnect()

# Connect / Disconnect
def toggle_connection(self) -> None:
    if not self.connected:
        try:
            username = self.user_entry.get().strip()
            host = self.host_entry.get().strip()
            port = int(self.port_entry.get().strip())

```

```

        if not username:
            raise ValueError("Username required")

        self.client.connect(host, port, username)

        self.connected = True
        self.connect_btn.config(text="Disconnect")
        self.msg_entry.config(state="normal")
        self.send_btn.config(state="normal")

        threading.Thread(
            target=self.listen_loop, daemon=True
        ).start()

        self.write_log("[SYSTEM]: Connected")

    except Exception as e:
        messagebox.showerror("Error", str(e))
    else:
        self.client.close()
        self.on_disconnect()

def on_disconnect(self) -> None:
    if not self.connected:
        return
    self.connected = False
    self.connect_btn.config(text="Connect")
    self.msg_entry.config(state="disabled")
    self.send_btn.config(state="disabled")
    self.write_log("[SYSTEM]: Disconnected")

def start(self) -> None:
    self.root.mainloop()

```

Server Logic

```

# =====
# SERVER LOGIC (Socket Programming)
# =====

import socket
import threading
from typing import Callable, List, Tuple, Optional

```

```

class ChatServer:
    """
    Core TCP chat server.

    Socket concepts used:
    - TCP (SOCK_STREAM)
    - Blocking sockets
    - Thread-per-client model
    """

    def __init__(self) -> None:
        self.server: Optional[socket.socket] = None

        # Each client is stored as (username, socket)
        self.clients: List[Tuple[str, socket.socket]] = []

        self.running = False

    def start(self, host: str, port: int, log_callback: Callable[[str], None]) -> None:
        # Create TCP socket
        self.server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        # Bind socket to IP and port
        self.server.bind((host, port))

        # Start listening for connections
        self.server.listen(5)

        self.running = True
        log_callback(f"[SERVER]: Listening on {host}:{port}")

        # Accept loop
        while self.running:
            try:
                # accept() blocks until a client connects
                conn, addr = self.server.accept()

                # First message from client is username
                username = conn.recv(1024).decode()

                # Save client
                self.clients.append((username, conn))

```

```

# Notify others
self.broadcast(f"[SYSTEM]: {username} joined the chat.", None)

log_callback(f"[SERVER]: {username} connected from {addr}")

# Handle client in separate thread
threading.Thread(
    target=self.handle_client,
    args=(username, conn, log_callback),
    daemon=True
).start()

except OSError:
    break

def handle_client(self, username: str, conn: socket.socket, log_callback) ->
None:
    while self.running:
        try:
            # recv() blocks waiting for data
            data = conn.recv(1024)

            # Empty recv means client closed connection
            if not data:
                break

            msg = data.decode()

            # Private message
            if msg.startswith("@"):
                parts = msg.split(" ", 1)
                if len(parts) > 1:
                    target = parts[0][1:]
                    content = parts[1]
                    self.send_private(target, username, content, conn)
            else:
                # Broadcast message
                log_callback(f"[{username}]: {msg}")
                self.broadcast(f"[{username}]: {msg}", conn)

        except:
            break

# Cleanup on disconnect
self.remove_client(username, conn, log_callback)

```

```

# Send to all except sender
def broadcast(self, message: str, sender_socket: Optional[socket.socket]) ->
None:
    for _, sock in self.clients:
        if sock != sender_socket:
            try:
                sock.sendall(message.encode())
            except:
                pass

# Send to a specific user
def send_private(self, target_user: str, sender_user: str, msg: str,
sender_sock):
    for user, sock in self.clients:
        if user == target_user:
            sock.sendall(
                f"[PRIVATE] {sender_user} → {target_user}: {msg}".encode()
            )
            return

# User not found
sender_sock.sendall(
    f"[SYSTEM]: User '{target_user}' not found.".encode()
)

def remove_client(self, username: str, conn: socket.socket, log_callback) ->
None:
    if (username, conn) in self.clients:
        self.clients.remove((username, conn))
        log_callback(f"[SERVER]: {username} disconnected")
        self.broadcast(f"[SYSTEM]: {username} left the chat.", conn)

    conn.close()

def stop(self) -> None:
    self.running = False

    if self.server:
        self.server.close()

    for _, conn in self.clients:
        try:
            conn.close()
        except:
            pass

```

```
self.clients.clear()
```

Client Logic

```
# =====
# CLIENT LOGIC (Socket Programming)
# =====

import socket
from typing import Optional

class ChatClient:
    """
    TCP client for chat application.

    Socket concepts:
    - Single persistent TCP connection
    - Blocking recv()
    """

    def __init__(self) -> None:
        self.sock: Optional[socket.socket] = None
        self.connected = False

    def connect(self, host: str, port: int, username: str) -> None:
        # Create TCP socket
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        # Establish TCP connection (3-way handshake)
        self.sock.connect((host, port))

        # Send username as application-level handshake
        self.sock.sendall(username.encode())

        self.connected = True

    def send(self, msg: str) -> None:
        if self.sock:
            self.sock.sendall(msg.encode())

    def receive(self) -> Optional[str]:
        if not self.sock:
            return None
```

```
    try:
        data = self.sock.recv(1024)
        if not data:
            return None
        return data.decode()
    except:
        return None

def close(self) -> None:
    self.connected = False
    if self.sock:
        self.sock.close()
        self.sock = None
```